```
Loading package containers-0. 5. 0. 0 ... linking ... done.
Loading package utf8-string-0. 3. 7 ... linking ... done.
Loading package mtl-2. 1. 2 ... linking ... done.
Loading package convertible-1. 1. 0. 0 ... linking ... done.
Loading package HDBC-2. 4. 0. 0 ... linking ... done.
Loading package HDBC-sqlite3-2. 3. 3. 0 ... linking ... done
```

The signature of the connectSqlite3 function is as follows:

```
ghci> : t connectSqlite3
connectSqlite3 :: FilePath -> IO Connection
```

The type of *conn* is a *Connection*.

```
ghci> : t conn
conn :: Connection
```

If you already have an existing Sqlite3 database, you can give the full path to the database and connect to it, or else you can create a table using the Sqlite *CREATE TABLE* syntax as shown below:

```
ghci> run conn "CREATE TABLE names (id INTEGER NOT NULL, fname
VARCHAR(80), lname VARCHAR(80))" []
0
```

The type signature of *run* is as follows:

```
ghci> : t run
run
:: IConnection conn => conn -> String -> [SqlValue] -> IO
Integer
```

It takes three arguments as input and performs an IO computation that returns an integer indicating the status of the execution. The first argument to run is the connection, the second argument is the Sqlite command to be executed, and finally there is the array of SqlValues that provide a mapping between Haskell values and SQL databases.

Both Haskell and SQL databases have types, and different databases may have different representations of the types. In order to provide a consistent mapping between the two, each HDBC driver implements the relation using SqlValue.

You can now insert a record into the database using the following command:

```
ghci> run conn "INSERT INTO names (id, fname, lname) VALUES(1,
'Edwin', 'Brady')" []
1
ghci> commit conn
```

The type signature of commit is given here:

```
ghci> : t commit
commit :: IConnection conn => conn -> IO ()
```

It takes a connection and completes the pending IO actions. To read the result from Haskell you can use the *quickQuery* function from the GHCi prompt, as follows:

```
ghci> quickQuery conn "SELECT * from names" []
[[SqlByteString "1", SqlByteString "Edwin", SqlByteString
"Brady"]]
```

The type signature of the *quickQuery* function is as follows:

```
quickQuery
:: IConnection conn =>
conn -> String -> [SqlValue] -> IO [[SqlValue]]
```

You can also verify the result of the above actions using the Sqlite3 executable in the command prompt as illustrated below:

```
$ sqlite3 students.db
SQLite version 3.8.4.3 2014-04-03 16:53:12
Enter ".help" for usage hints.
sqlite> .schema
CREATE TABLE names (id INTEGER NOT NULL, fname VARCHAR(80),
lname VARCHAR(80));
sqlite> select * from names;
1|Edwin|Brady
```

You can also do batch processing for inserts by preparing the statements and executing them:

```
ghci> batch <- prepare conn "INSERT INTO names VALUES (?, ?,
?)"
ghci> execute batch [toSql (2 :: Int), toSql "Simon", toSql
"Marlow"]
1
ghci> execute batch [toSql (3 :: Int), toSql "Ulf", toSql
"Norell"]
1
ghci> commit conn
```

The type signatures of the *prepare* and *execute* functions are given below:

```
ghci> : t prepare
prepare :: IConnection conn => conn -> String -> IO Statement
ghci> : t execute
execute :: Statement -> [SqlValue] -> IO Integer
```

You can once again check the records in the database using the *quickQuery* function:

```
ghci> quickQuery' conn "SELECT * from names" []
[[SqlByteString "1", SqlByteString "Edwin", SqlByteString
"Brady"],[SqlByteString "2", SqlByteString "Simon",
SqlByteString "Marlow"],[SqlByteString "3", SqlByteString
```